

Project Retrospective

RecallGuard — Conversational Memory Intelligence System

Deliverable 7 | Engineering Journal and Project Retrospective | August 2026

1. Project Overview

RecallGuard is a conversational memory system designed to persist, retrieve, rank, maintain, and provide relevant memories for user interactions. The project evolved from the earlier deliverables into a persistent ChromaDB-backed implementation with a modular application architecture.

The final implementation separates persistence, retrieval, ranking, context construction, reflection, authentication, logging, and the Flask API. This separation was important for testing individual responsibilities and for keeping infrastructure details out of higher-level components.

2. What Was Implemented

Component	Purpose
MemoryStore	Persistent ChromaDB storage, retrieval, updates, deletion, duplicate detection, and expiration.
MemoryRetriever	Embedding-based semantic retrieval with memory-type and confidence filtering.
MemoryRanker	Ranking retrieved memories before they are used by downstream components.
ContextBuilder	Constructing relevant context from retrieved memories.
ReflectionEngine	Memory summarization and maintenance such as expiration, duplicates, and low-confidence cleanup.
Authentication	API-key authentication and role-aware access control.
MemoryLogger	Structured application, retrieval, security, reflection, performance, and error logging.
Flask API	HTTP endpoints for storing, retrieving, contextualizing, maintaining, listing, and deleting memories.

3. Major Engineering Decisions

- **ChromaDB as the vector store.** ChromaDB provided persistent vector storage together with metadata filtering, which matched the project's semantic retrieval requirements.
- **MemoryStore as the persistence boundary.** Higher-level modules access memory through MemoryStore rather than directly depending on ChromaDB.
- **User isolation at query time.** Retrieval applies the user identifier as part of the ChromaDB filter, reducing the risk of cross-user memory exposure.
- **Independent test collections.** Tests use separate ChromaDB collections so persistent data from earlier executions cannot contaminate later tests.
- **Separate retrieval policy.** MemoryRetriever handles embedding-based retrieval and application-level filtering while MemoryStore handles persistence.
- **Reflection as a separate maintenance layer.** Expiration, duplicate removal, and low-confidence cleanup are kept outside the core storage implementation.

4. Important Failures and Debugging

- **Configuration mismatch.** An early memory-store implementation expected `DATABASE_PATH`, while the project configuration exposed `CHROMA_DB_PATH`. The storage layer was aligned with the actual ChromaDB configuration.
- **Global collection dependency.** Reflection initially referenced an undefined collection variable. This was corrected by using the database collection abstraction.
- **Confidence-filter test failure.** A test unexpectedly returned older high-confidence memories. Investigation showed that persistent test data was being reused. Test fixtures were changed to create unique collections.
- **User-isolation test failure.** Cross-user retrieval required filtering at the persistence boundary. The ChromaDB query was updated to include `user_id` and `deleted-state` filters.
- **Missing dependency.** The Flask application initially failed because `sentence-transformers` was not installed. The dependency was added to the project's requirements.
- **Root URL returned 404.** The application was running correctly, but no `/` route had been defined. The existing `/health` endpoint was used as the service health check.

5. Testing and Verification

The retrieval test suite covered basic retrieval, top-k limits, preference filtering, fact filtering, task filtering, confidence filtering, user isolation, empty searches, and similarity ordering. The final failures were resolved by improving collection isolation and enforcing user-level filtering.

The Flask application was also started successfully, the embedding model loaded, and the API was exercised using API-key authentication. These checks provided confidence that the components worked together beyond isolated unit tests.

6. What Worked Well

- Separating storage from retrieval made the system easier to reason about.
- ChromaDB reduced the amount of infrastructure needed for persistent vector retrieval.
- Metadata filtering provided a straightforward mechanism for user isolation and memory-type filtering.
- Tests exposed problems that were not obvious from code inspection, especially persistent test contamination.
- Structured logging created an observability layer around important memory operations.

7. What Could Be Improved

- The test environment should use a clearly isolated database lifecycle from the beginning.
- Dependencies should be verified through a clean environment before runtime testing.
- The Flask application could expose a simple root endpoint in addition to `/health` for easier manual verification.
- Authentication currently uses in-memory API keys; production deployment would require a secure persistent identity and key-management mechanism.
- The reflection and ranking policies could be evaluated with larger, more representative datasets.

8. Engineering Lessons

The most important lesson was to distinguish observation, inference, and decision during debugging. A failing test does not necessarily mean the immediately suspected function is incorrect. In this project, a retrieval failure led to investigation of persistent test data and collection isolation before changing the retrieval algorithm.

Another important lesson was the value of stable abstraction boundaries. By keeping ChromaDB behind MemoryStore, changes to collection handling and test setup did not require rewriting every component.

9. Final Project State

Deliverable 6 was completed with the core conversational-memory implementation, API integration, authentication, logging, reflection, and retrieval tests. Deliverable 7 documents the engineering reasoning through dated journal entries and architecture decision records.

The project is therefore at a point where the next stage should focus on deeper integration testing, production hardening, stronger security and secrets management, performance evaluation, and validation with realistic conversational workloads.

Retrospective conclusion: The project progressed from individual memory components to an integrated persistent conversational-memory service. The debugging process was as important as the implementation itself, because it revealed the importance of isolation, abstraction boundaries, and explicit verification.